

# Pi-CloudDOOM — CI/CD Pipeline Report

**Repo:** <https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM> **Workflows:** `.github/workflows/` (13 files) **Docs:** `docs/cicd/` (step-00 through step-08 + this report)

---

## 1. Goal

Every commit follows a single, automated path from `git push` to live production:

`git push`   CI (lint/test/build)   Build & sign images   Deploy to 3 planes   Smoke   Done

Code scanning, IaC scan, secret scan, dependency PRs

Zero manual `kubectl`, `az containerapp`, or `vercel deploy` invocations in the normal flow. All deploys are auditable in the Actions tab and reversible via `workflow_dispatch`.

---

## 2. Pipeline architecture

PR opened   CI per-service   paths-filter → reusable wfs  
CodeQL · IaC scan · Gitleaks  
Vercel preview URL on PR (sticky comment)

Merge to main / push to deployment/\*\*

Build & Push images (matrix × 10)  
Buildx + GHA cache → DockerHub  
Trivy scan (report-only day-1)  
Cosign keyless signing (OIDC)  
Tags: sha-<short>, branch, latest

Deploy - OpenStack K8s  
matrix × 7 Spring svcs  
`kubectl set image ...`  
`rollout status` (10m)

Deploy - Azure ACA  
matrix × 3 AI svcs  
`az containerapp update ...`  
revision health poll

→ Smoke test (in-cluster) → auto-rollback if Failed  
 → Auto-rollback on red

Push to main (frontend/\*\*) Vercel - Production  
 prod URL: <https://interviewprep-tn.me>

### 3. Workflow index (13 files)

| File                  | Trigger                                                               | Purpose                                                                                                                                  |
|-----------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| ci.yml                | PR + push to <b>main</b> ,<br><b>deployment/**</b>                    | Orchestrator —<br>paths-filter then<br>dispatch to reusable<br>per-service CI workflows.<br>Ends in <b>ci-summary</b><br>aggregate gate. |
| ci-spring-service.yml | reusable                                                              | Maven <b>verify</b><br><b>-DskipITs</b> per Spring<br>service. Uploads<br>Surefire reports on<br>failure.                                |
| ci-python-service.yml | reusable                                                              | <b>pip install</b> +<br><b>compileall</b> + <b>import</b><br><b>app</b> smoke + pytest if<br>tests exist.                                |
| ci-frontend.yml       | reusable                                                              | <b>npm ci</b> + <b>ng build</b><br><b>--configuration</b><br><b>production</b> . Uploads<br>dist on <b>main</b> .                        |
| build-and-push.yml    | push to<br><b>main/deployment/**</b> ,<br>tags <b>v*.*.*</b> , manual | Matrix builds 10 images,<br>Trivy scan, Cosign<br>keyless sign, push to<br>DockerHub.                                                    |
| deploy-k8s.yml        | <b>workflow_run</b> after<br>build (auto-fires on<br>success), manual | Rolls 7 Spring services<br>into <b>piclouddoom</b> (or<br><b>piclouddoom-dev</b> ) +<br>smoke + rollback.                                |
| deploy-aca.yml        | manual (auto-trigger<br>disabled pending Azure<br>SP)                 | Rolls 3 AI services into<br>Azure Container Apps +<br>health poll +<br>traffic-shift rollback.                                           |
| vercel-preview.yml    | PR touching<br><b>frontend/**</b>                                     | Preview deploy + sticky<br>PR comment with URL.                                                                                          |

| File            | Trigger                              | Purpose                                                                             |
|-----------------|--------------------------------------|-------------------------------------------------------------------------------------|
| vercel-prod.yml | push to main touching<br>frontend/** | Production deploy + 5×<br>retry smoke check.                                        |
| codeql.yml      | PR + push + Monday<br>05:00 UTC      | SAST across<br>java-kotlin,<br>javascript-typescript,<br>python, actions.           |
| iac-scan.yml    | PR + push to k8s/**<br>or infra/**   | Trivy config +<br>kube-linter on manifests<br>(report-only).                        |
| gitleaks.yml    | PR + push + Monday<br>05:30 UTC      | Full-history secret scan<br>with <code>fetch-depth: 0</code> .                      |
| dependabot.yml  | scheduled                            | Weekly grouped dep<br>PRs (Maven × 7, npm,<br>pip, Docker × 11,<br>GitHub Actions). |

## 4. Continuous Integration (CI)

### 4.1 Smart per-service CI — ci.yml

The orchestrator uses `dorny/paths-filter@v3` to compute, per PR/push, which folders changed and only fires the relevant per-service jobs:

```
changes:
  steps:
    - uses: dorny/paths-filter@v3
      with:
        filters: |
          user:      ['user-service/**']
          interview: ['interview-service/**']
          training:  ['training-service/**']
          mentorship: ['mentorship-service/**']
          quiz:      ['quiz-service/**']
          community: ['community-service/**']
          resource:  ['resource-service/**']
          ai:        ['ai-training-path/**']
          frontend:  ['frontend/**']
```

Each filter output gates a downstream job that calls a **reusable workflow** with the right parameters:

- Spring services → `ci-spring-service.yml` (Java 21 default; `quiz-service` overrides to Java 17 because it still runs Spring Boot 3.4.1).
- `ai-training-path` → `ci-python-service.yml` (Python 3.12).

- frontend → ci-frontend.yml (Node 20).

## 4.2 Why reusable workflows?

Three reusable workflows (one per language family) keep CI logic in **one place** and the orchestrator dead simple. Adding a new Spring service is just two lines in `ci.yml`. Java version, service folder, and (for Python) python version are passed as `with:` inputs.

## 4.3 Aggregate gate — ci-summary

A single job depends on **all** per-service jobs (`needs: [user, interview, training, mentorship, quiz, community, resource, ai, frontend, changes]`) and uses Python to inspect the `needs` JSON, failing only if any required job ended in **failure**. This is the single status check we wire into branch protection — reviewers don't have to tick 9 boxes.

## 4.4 What each per-service CI does

| File                  | Stack  | Steps                                                                                                                                                        |
|-----------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ci-spring-service.yml | Java   | setup-java (Temurin,<br>Maven cache) → mvn<br>-B -ntp -DskipITs<br>verify → upload<br>Surefire reports on<br>failure                                         |
| ci-python-service.yml | Python | setup-python (pip<br>cache) → pip install<br>-r requirements.txt<br>→ python -m<br>compileall -q . →<br>python -c "import<br>app" → pytest if tests<br>exist |
| ci-frontend.yml       | Node   | setup-node (npm<br>cache) → npm ci<br>--no-audit --no-fund<br>→ npx ng build<br>--configuration<br>production → upload<br>dist artifact on main              |

## 4.5 Concurrency

`concurrency.group: ci-${{ github.workflow }}-${{ github.ref }}`  
 with `cancel-in-progress: true` — pushing 3 commits in 10 seconds keeps

only the latest run.

---

## 5. Image build, scan, sign, push — build-and-push.yml

### 5.1 Triggers

- Push to main or deployment/\*\*.
- v\*.\*.\* git tag.
- workflow\_dispatch (manual; optional service: input to build one).
- paths-ignore skips builds for doc-only, frontend-only, and .github/\*\* changes.

### 5.2 The matrix (10 jobs)

```
matrix:
  include:
    - { service: user-service,      image: azizbna/pi-clouddoom-user-service      }
    - { service: interview-service, image: azizbna/pi-clouddoom-interview-service }
    - { service: training-service,  image: azizbna/pi-clouddoom-training-service  }
    - { service: mentorship-service, image: azizbna/pi-clouddoom-mentorship-service }
    - { service: quiz-service,      image: azizbna/pi-clouddoom-quiz-service      }
    - { service: community-service, image: azizbna/pi-clouddoom-community-service }
    - { service: resource-service,  image: azizbna/pi-clouddoom-resource-service  }
    - { service: ai-training-path,  image: azizbna/pi-clouddoom-ai-training-path  }
    - { service: kokoro,            image: azizbna/pi-clouddoom-kokoro            }
    - { service: ollama-deploy,     image: azizbna/pi-clouddoom-ollama            }
```

fail-fast: false → one broken service doesn't sink the other 9.

### 5.3 Per-job steps

1. **Skip-if-not-matched** — supports the single-service workflow\_dispatch service: input.
2. **docker/setup-buildx-action@v3** — BuildKit + multi-platform readiness.
3. **docker/login-action@v3** with DOCKERHUB\_USERNAME + DOCKERHUB\_TOKEN.
4. **docker/metadata-action@v5** — generates the full tag set:

```
tags: |
  type=sha,prefix=sha-,format=short
  type=ref,event=branch
  type=ref,event=tag
  type=semver,pattern={{version}}
  type=raw,value=latest,enable={{is_default_branch}}
```

5. **docker/build-push-action@v6:**
  - **context:** `./<service>`
  - **provenance:** `mode=max` → SLSA build provenance attached.
  - **sbom:** `true` → SBOM attached.
  - **cache-from:** `type=gha,scope=<service>` / **cache-to:** `...,mode=max` → per-service GHA cache ( 30s for unchanged builds).
6. **aquasecurity/trivy-action@v0.36.0** — scans the just-built image by digest:
  - **severity:** `CRITICAL,HIGH`, **ignore-unfixed:** `true`.
  - **exit-code:** `"0"` (report-only day-1; flip to `"1"` after backlog triage).
  - SARIF uploaded as artifact.
7. **sigstore/cosign-installer@v3** → **cosign sign --yes** — keyless OIDC signing. No private key.
8. **Step summary** — image, digest, and tag list in the GitHub Actions step summary.

#### 5.4 Aggregate gate — summary job

Single job depends on the matrix; fails if any service build failed. Hooks into branch protection.

---

## 6. Backend deploy — deploy-k8s.yml

### 6.1 Triggers

- **workflow\_run** on Build & Push images completing successfully on `main` or `deployment/hybrid-openstack-azure-vercel`. Auto-fires; never on a failed build (the `gate` job checks `conclusion == 'success'`).
- **workflow\_dispatch** with inputs:
  - **environment** (choice: `dev`, `prod`)
  - **sha** (optional — for rolling back to a specific commit)
  - **service** (optional — deploy only one of the 7)

### 6.2 The gate job

Resolves three things from the trigger: - **SHA:** from `workflow_run.head_sha` or manual input → `sha-<7-char>`. - **Environment:** `workflow_run` → always `prod`; `workflow_dispatch` → user-selected. - **Namespace:** `piclouddoom` (`prod`) / `piclouddoom-dev` (`dev`).

These outputs feed the matrix.

### 6.3 The deploy matrix (7 jobs)

Per Spring service:

1. **Skip-if-not-matched** (single-service dispatch).
2. **azure/setup-kubectl@v4** pinned to v1.29.15 (matches cluster).
3. **Configure kubeconfig** from KUBECONFIG\_PROD or KUBECONFIG\_DEV secret based on `gate.outputs.environment`.
4. **kubectl set image deployment/<svc> <svc>=<registry>:sha-<short> --record=false**.
5. **kubectl rollout status deployment/<svc> --timeout=10m** (bumped from 5 m → 10 m; `resource-service` JVM warmup measured at ~142 s, the original 5-min budget was too tight once readiness + dependency probes stacked).
6. **On rollout failure:** `describe deployment/<svc>, get pods -l app=<svc>`, and last 80 log lines per pod for the audit log.
7. **Auto-rollback:** `kubectl rollout undo deployment/<svc> + re-wait 3 min`.
8. **Step summary:** image pinned, rollout exit code, pod listing.

The job attaches to a **GitHub Environment** (production for prod, dev for dev) — letting us put required reviewers on prod from the repo settings UI without touching YAML.

### 6.4 The smoke job

After the matrix succeeds:

1. Re-configures kubeconfig (each job runs on a fresh runner).
2. Runs `bash scripts/smoke.sh --kubeconfig=$HOME/.kube/config --namespace=<ns>`.

`smoke.sh` has **two modes**: - **HTTP** (default): `curl $BASE_URL/actuator/health` for each of the 7 services. Tests the full Cloudflared → Traefik → Service path. - **kubectl** (`--kubeconfig + --namespace`): `kubectl wait --for=condition=Available --timeout=Ns deployment/<svc> + verifies status.readyReplicas >= 1`. Used by CI because the Spring containers run on `eclipse-temurin:21-jre` (no curl). `Available=True` exactly means the manifest's readiness probe (`httpGet /actuator/health`) is passing — same validation, no in-container curl.

### 6.5 Smoke-fail → namespace-wide rollback

If smoke fails, an `if: failure()` step rolls back **all 7 services**, then re-waits each to converge. The bad image stays in the registry (immutable) but is no longer pinned to any deployment.

## 6.6 Aggregate gate

deploy-summary checks `deploy.result` AND `smoke.result` — both must be green.

## 6.7 Concurrency

group: `deploy-k8s-${ inputs.environment || 'prod' }` with `cancel-in-progress: false` — never kill a rollout mid-flight, but dev and prod can deploy simultaneously.

---

## 7. AI deploy — `deploy-aca.yml`

### 7.1 Trigger note

`workflow_run` auto-trigger is **commented out** (header note in the file, decided 2026-05-11) because `AZURE_CREDENTIALS` can't be created until the `esprit.tn` Entra ID admin grants the *Application Developer* role. Until then: - Workflow runs only via `workflow_dispatch` (manual). - `scripts/deploy-aca.sh` is the laptop fallback — same logic, run with a developer's `az login`. Once the SP exists, uncomment the `workflow_run` block and CI takes over.

### 7.2 The matrix (3 jobs)

| Source            | ACA app          | Image                                           |
|-------------------|------------------|-------------------------------------------------|
| ai-training-path/ | ai-training-path | docker.io/azizbna/pi-clouddoom-ai-training-pat. |
| kokoro/           | kokoro           | docker.io/azizbna/pi-clouddoom-kokoro           |
| ollama-deploy/    | ollama           | docker.io/azizbna/pi-clouddoom-ollama           |

**whisper not in the matrix** — uses third-party `fedirz/faster-whisper-server:latest-cpu`, we don't build it.

### 7.3 Per-job steps

1. `azure/login@v2` with `AZURE_CREDENTIALS` JSON.
2. `az containerapp show` — log pre-update state (image, latestRev).
3. `az containerapp update --image <ref>:sha-<short> --revision-suffix sha-<short>` — creates a new revision.
4. **Health poll** ( $30 \times 10\text{ s} = 5\text{ min max}$ ) of `properties.runningState`:
  - `Running` / `RunningAtMaxScale` → success.
  - `Failed` / `Degraded` → exit 1.
  - `Timeout` → exit 1.



5. **Auto-rollback** (if: failure()):

```
PREV_REV=$(az containerapp revision list ... \
  --query "sort_by([?properties.runningState=='Running' && name!='<broken>'], &properties.creationTime)" | jq -r '.[0].name')
az containerapp ingress traffic set ... --revision-weight "$PREV_REV=100"
```

6. **Step summary** with image + revision name + post-deploy fqdn.

## 7.4 Concurrency

group: deploy-aca-prod with cancel-in-progress: false.

---

## 8. Frontend deploys — vercel-preview.yml + vercel-prod.yml

### 8.1 Preview (vercel-preview.yml)

- Trigger: PR touching frontend/\*\*, vercel.json, .vercel/\*\*, .vercelignore, or the workflow file itself.
- Steps: install Node 20, npm i -g vercel@latest, vercel pull --environment=preview, vercel build, vercel deploy --prebuilt, **sticky PR comment** with the preview URL via marocchino/sticky-pull-request-comment@v2.
- Concurrency keyed on PR number → 3 commits in a row only keep the latest preview.

### 8.2 Production (vercel-prod.yml)

- Trigger: push to main touching the same paths, or workflow\_dispatch.
- Steps: same flow with --environment=production + --prod flags.
- **Post-deploy smoke:** curl https://interviewprep-tn.me 5× with 10 s retries. Job fails on no-200; the existing prod deployment continues serving (Vercel won't replace until new is live).
- Attached to GitHub Environment production with url: https://interviewprep-tn.me.
- cancel-in-progress: false — never kill a prod deploy mid-flight.

### 8.3 Public env vars (in YAML, not secrets)

```
VERCEL_ORG_ID:      team_tQYcRewNfvSBGAbsE6wJwcp
VERCEL_PROJECT_ID:  prj_2vh15NTOTAWAQZkKNWPm3UKs25HL
```

Already public in the committed .vercel/project.json — not sensitive.

---

## 9. Security gates

### 9.1 CodeQL — `codeql.yml`

- 4 parallel analyses in a matrix: `java-kotlin`, `javascript-typescript`, `python`, `actions`.
- Queries: `security-extended`, `security-and-quality` (broader than default).
- For Java: `autobuild` then `mvn -B -ntp -DskipTests package` in `user-service/` (representative Spring service with tests, on Java 21).
- Schedule: PR + push + cron: `"0 5 * * 1"` (Mondays 05:00 UTC) — catches newly-disclosed CVE patterns against the old code.
- Findings → GitHub Security → Code scanning alerts.

### 9.2 IaC scan — `iac-scan.yml`

Two complementary tools:

| Tool                | Strength                                                                                                                                                |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trivy config</b> | K8s anti-patterns (missing limits, <code>:latest</code> tags, privileged containers, <code>hostPath</code> , ...), Heat / Terraform / Dockerfile checks |
| <b>kube-linter</b>  | Production-readiness (probes, replica count, anti-affinity, PDBs)                                                                                       |

Both upload SARIF to GitHub code-scanning under their own categories (`trivy-k8s`, `trivy-infra`, `kube-linter`). **Report-only** for day-1 (`exit-code: "0"` on Trivy, `continue-on-error: true` on kube-linter) — flip to enforcing once findings are triaged.

Triggers: PR + push touching `k8s/**` or `infra/**`.

### 9.3 Gitleaks — `gitleaks.yml`

- **Full-history scan:** `fetch-depth: 0` so secrets buried in old commits get caught.
- Triggers: PR + push + Mondays 05:30 UTC.
- Findings → SARIF + step summary + artifact.
- Uses auto-provisioned `GITHUB_TOKEN`; no extra secret.

### 9.4 In-image scan

Already part of `build-and-push.yml` — Trivy scans every produced image. SARIF uploaded as artifact (`trivy-<service>`).

## 9.5 Supply chain

- **Cosign keyless signing** in `build-and-push.yml` — verifiable by any developer with `cosign verify`.
- **SBOM + build provenance** attached as OCI attestations.
- **Dependabot** weekly across Maven (7), npm (1), pip (1), Docker (11), GitHub Actions (1) — grouped by family (Spring core, Angular core, test libs) to reduce PR noise.

---

## 10. GitHub secrets

| Secret               | Used by                                                   | Source                                                                      |
|----------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------|
| DOCKERHUB_USERNAME   | <code>build-and-push.yml</code>                           | <code>azizbna</code>                                                        |
| DOCKERHUB_TOKEN      | <code>build-and-push.yml</code>                           | <code>https://hub.docker.com/settings/security (R/W/D)</code>               |
| VERCEL_TOKEN         | <code>vercel-preview</code> ,<br><code>vercel-prod</code> | <code>https://vercel.com/account/tokens (Full Account)</code>               |
| KUBECONFIG_PROD      | <code>deploy-k8s.yml (prod path)</code>                   | <code>scripts/extract-kubeconfig.sh on k8s-cp1</code>                       |
| KUBECONFIG_DEV       | <code>deploy-k8s.yml (dev path)</code>                    | <code>NAMESPACE=picloudboom-dev scripts/extract-kubeconfig.sh</code>        |
| AZURE_CREDENTIALS    | <code>deploy-aca.yml</code>                               | <code>az ad sp create-for-rbac --sdk-auth JSON (pending Entra grant)</code> |
| AZURE_RESOURCE_GROUP | <code>deploy-aca.yml</code>                               | <code>rg-picloudboom (plain string, kept secret by choice)</code>           |

Nothing else is needed — CodeQL, IaC scan, and Gitleaks use the auto-provisioned `GITHUB_TOKEN`.

---

## 11. GitHub Environments

Configured under **Settings → Environments**:

| Environment             | Used by                                                                                   | Recommended protections                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>production</code> | <code>deploy-k8s (prod)</code> ,<br><code>deploy-aca</code> ,<br><code>vercel-prod</code> | 1 required reviewer, restrict to <code>main + deployment/hybrid-openstack-azure-vercel</code> |

| Environment | Used by          | Recommended protections |
|-------------|------------------|-------------------------|
| dev         | deploy-k8s (dev) | None                    |

production is also where you'd add environment-scoped secrets if anything diverged between dev and prod beyond kubeconfigs.

## 12. Branch protection (recommended on main)

Use the `ci-summary`, `build-summary`, `deploy-summary` aggregate jobs as required status checks:

- `ci-summary` — must be green before merge.
- CodeQL analysis (per language).
- Gitleaks scan.
- (optional) IaC scan once it's not report-only.

This keeps branch-protection rules small — one check per pipeline stage.

## 13. Pipeline observability

Every workflow writes a `$GITHUB_STEP_SUMMARY` with: - Image + digest + tag list (`build-and-push.yml`) - Image pinned + rollout exit code + pod listing (`deploy-k8s.yml`) - Image + revision name + post-deploy fqdn (`deploy-aca.yml`) - Deployment URL + commit (`vercel-prod.yml`)

So a developer opening the run sees the whole picture without reading 800 lines of log.

## 14. The 9 build steps (history)

The pipeline was built incrementally in 9 commits, one per “step”:

| # | Step                                                           | Commit   | Doc                     |
|---|----------------------------------------------------------------|----------|-------------------------|
| 0 | Repo hygiene (dockerignores, move heat-cluster.yaml into repo) | c515dec9 | step-00-repo-hygiene.md |

| # | Step                                                             | Commit   | Doc                         |
|---|------------------------------------------------------------------|----------|-----------------------------|
| 1 | CI scaffolding<br>(paths-filter,<br>reusable wfs,<br>dependabot) | 8fc603f2 | step-01-ci-scaffolding.md   |
| 2 | Image build &<br>push (matrix ×<br>10, Trivy,<br>Cosign)         | bb03d129 | step-02-image-build-push.md |
| 3 | Vercel preview +<br>prod pipelines                               | 18aa2fdf | step-03-vercel-pipeline.md  |
| 4 | K8s deploy +<br>RBAC +<br>kubeconfig<br>extraction               | c685dc72 | step-04-k8s-deploy.md       |
| 5 | Azure Container<br>Apps deploy                                   | a7fa4b10 | step-05-aca-deploy.md       |
| 6 | Security gates<br>(CodeQL, IaC<br>scan, Gitleaks)                | 2bfd6e2f | step-06-security-gates.md   |
| 7 | Multi-env<br>(dev/prod) +<br>smoke tests                         | f7096c44 | step-07-multi-env-smoke.md  |
| 8 | Finalize docs +<br>runbook                                       | 27de2227 | step-08-finalize.md         |

Each step's \*.md doc explains the file, the design choice, the OpenStack actions needed (if any), known caveats, and verification steps — together they form an executable runbook for re-creating the pipeline from scratch.

## 15. Anatomy of one full release (end-to-end)

- Developer pushes to `deployment/hybrid-openstack-azure-vercel`.
- CI workflows fire IN PARALLEL:
  - ci.yml (lint/test/build for changed services)
  - codeql.yml (SAST per language)
  - iac-scan.yml (if k8s/ or infra/ changed)
  - gitleaks.yml (secret scan)
- CI passes → all aggregate jobs go green.
- `build-and-push.yml` fires (push-triggered, doesn't wait for CI):

- Matrix builds 10 images in parallel
  - Each: build → Trivy → cosign sign → push (sha-<short>, branch, latest if main)
5. ``build-and-push.yml`` finishes successfully.
  6. ``deploy-k8s.yml`` auto-fires via `workflow_run`:
    - gate resolves sha-<short> + namespace=piclouddoom
    - 7 rollout jobs in parallel: `kubectl set image` + rollout status (10 min)
    - smoke job: `kubectl wait Available + readyReplicas>=1` on each Deployment
    - deploy-summary: green only if both deploy and smoke green
  7. (When unblocked) ``deploy-aca.yml`` auto-fires:
    - 3 revision updates in parallel
    - Each: `az containerapp update` + 5-min health poll
    - Auto-traffic-shift back on failure
  8. If frontend changed → ``vercel-prod.yml`` fires on push to main:
    - `vercel pull/build/deploy --prod`
    - `curl https://interviewprep-tn.me` 5× retry
  9. Done. Every step is in the Actions tab with a step summary.

---

## 16. Failure modes & their automatic responses

| Failure                                       | Detected by                                | Automatic response                                                                                    |
|-----------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Compile error / unit test fail                | <code>ci.yml</code> reusable wf            | CI gate fails → branch protection blocks merge                                                        |
| Image build fails                             | <code>build-and-push.yml</code> job        | Matrix continues for other services; aggregate gate fails; no deploy fires                            |
| Trivy CVE found (when enforced)               | <code>build-and-push.yml</code> Trivy step | Build fails → no signed/pushed image                                                                  |
| Cosign sign fails                             | <code>build-and-push.yml</code>            | Build fails → push has happened but image is unsigned (caught by verify step in cluster, post-launch) |
| <code>kubectl rollout status</code> times out | <code>deploy-k8s.yml</code> rollout step   | Logs dumped → <code>kubectl rollout undo</code> → re-wait 3 min                                       |
| Smoke test fails                              | <code>deploy-k8s.yml</code> smoke job      | Roll back <b>all 7</b> services in the namespace                                                      |

| Failure                         | Detected by                                | Automatic response                                                   |
|---------------------------------|--------------------------------------------|----------------------------------------------------------------------|
| ACA revision<br>Failed/Degraded | <code>deploy-aca.yml</code> health<br>poll | Traffic-shift 100% back<br>to previous Running<br>revision           |
| Vercel prod curl 200            | <code>vercel-prod.yml</code><br>smoke      | Job fails — but Vercel<br>keeps the previous prod<br>deployment live |
| Secret committed                | <code>gitleaks.yml</code>                  | PR check fails, finding<br>shown in Security tab                     |
| K8s manifest misconfig          | <code>iac-scan.yml</code>                  | SARIF posted, Security<br>tab populated (advisory<br>today)          |

## 17. Manual escape hatches

Everything automatic also has a manual switch:

| Operation                       | How                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Re-build one service            | Actions → “Build & Push images” →<br>Run workflow → <code>service:</code><br><code>user-service</code>                                                                                                                                                                                                                               |
| Re-deploy one service to a SHA  | Actions → “Deploy — OpenStack<br>K8s” → <code>service:</code> <code>user-service</code> ,<br><code>sha:</code> <7-char>                                                                                                                                                                                                              |
| Roll back all services to a SHA | “Deploy — OpenStack K8s” → leave<br><code>service:</code> blank, set <code>sha:</code> to the old<br>commit                                                                                                                                                                                                                          |
| Manual ACA deploy from a laptop | <code>bash scripts/deploy-aca.sh</code><br>[sha] [app] (uses <code>az login</code> )                                                                                                                                                                                                                                                 |
| Mint a new kubeconfig           | <code>bash</code><br><code>scripts/extract-kubeconfig.sh</code> ><br>kc.yaml on k8s-cp1, paste, shred                                                                                                                                                                                                                                |
| Emergency rollback w/o GitHub   | SSH to k8s-cp1 → <code>kubectl -n</code><br><code>picloudboom rollout undo</code><br><code>deployment/&lt;svc&gt;</code>                                                                                                                                                                                                             |
| Verify a signed image           | <code>cosign verify</code><br><code>docker.io/azizbna/pi-cloudboom-&lt;svc&gt;:sha-&lt;short&gt;</code><br><code>--certificate-identity-regex</code><br><code>"https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM/.*</code><br><code>--certificate-oidc-issuer</code><br><code>https://token.actions.githubusercontent.com</code> |

## 18. Day-1 known limitations (recap)

1. IaC scan + image scan are report-only.
  2. ACA auto-trigger disabled until Azure SP is approved.
  3. Smoke covers only the 7 Spring services (extend to ACA + Vercel post-launch).
  4. Single replica per app — rolling update is binary.
  5. No DB-migration coordination (Flyway runs at boot).
  6. No GitOps (Argo CD is on the backlog).
  7. CodeQL Java analyses only **user-service** (representative module).
  8. Service principal expires yearly → migrate to federated OIDC when possible.
- 

## 19. Summary

| Question                                        | Answer                                                                                                      |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| How many workflows?                             | 13 (5 reusable/orchestrators + 4 deploy + 3 security + 1 dependabot)                                        |
| Where do images live?                           | DockerHub<br><code>docker.io/azizbna/pi-clouddoom-*</code><br>(10 public repos)                             |
| What ties build → deploy?                       | <code>workflow_run</code> after successful <code>Build &amp; Push images</code>                             |
| What guarantees image integrity?                | Cosign keyless OIDC + SBOM + provenance + Trivy scan SARIF                                                  |
| What guarantees rollback?                       | Immutable <code>sha-&lt;short&gt;</code> tags + auto-undo on rollout/smoke fail + traffic-shift on ACA fail |
| What kicks off a prod deploy?                   | A successful image build on <code>main</code> or <code>deployment/hybrid-openstack-azure-vercel</code>      |
| What can you do without <code>git push</code> ? | Every workflow has a <code>workflow_dispatch</code> — one-click manual run with optional inputs             |
| Where do I see security findings?               | GitHub Security tab → Code scanning alerts (CodeQL, trivy-k8s, trivy-infra, kube-linter)                    |
| Where do I see deploy history?                  | GitHub Actions tab + Environments tab (production / dev / preview)                                          |